SIMATS Engineering

ASSIGNMENT 3 – CO3 – Part 2 (Q5–Q7)

Course Code	DSA0104
Course Name	Object Oriented Programming with C++
Topic	Constructors, Destructors and Operator Overloading (CO3)
Nature of Assignment	Individual
Total Marks	50
Bloom's Level	K5 – Evaluate
Student Name	Sangeet T R
Register Number	1921260245
GitHub Repository	https://github.com/Sangeetthangaraj/DSA0104---object-oriented-programming-with-c-
Submission Date	05/11/26

Contents

Q5 Increment, Comparison and Subscript (6) · Q6 Rules of Operator Overloading (7) · Q7 Complex-Number Class (10)

Build command used for every program: `g++ -std=c++17 -Wall -Wextra -g file.cpp -o out` (add `-fsanitize=address` where stated). Compiler: g++ ____ (fill in version, e.g. 13).

Note: Output blocks marked *Expected output* are what the code prints. Paste your own screenshot under each one after running, and fill every `_____` in the timing tables with your own measurements.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

Implementation (`Task05/playlist.cpp`)

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time {
    int sec;   // total seconds
public:
    Time(int s = 0) : sec(s) {}
    Time& operator++()    { ++sec; return *this; }                 // prefix
    Time  operator++(int) { Time old = *this; ++sec; return old; } // postfix
(dummy int)
    bool operator==(const Time& o) const { return sec == o.sec; }
    bool operator<(const Time& o)  const { return sec <  o.sec; }
    friend std::ostream& operator<<(std::ostream& os, const Time& t) {
        return os << t.sec / 3600 << ':' << std::setfill('0') << std::setw(2)
<< (t.sec / 60) % 60
                  << ':' << std::setw(2) << t.sec % 60 << std::setfill(' ');
    }
};
class Playlist {
    std::vector<std::string> tracks;
public:
    void add(const std::string& t) { tracks.push_back(t); }
    int size() const { return static_cast<int>(tracks.size()); }
    std::string& operator[](int i) {
        if (i < 0 || i >= size()) { std::cerr << "Index " << i << " out of
range\n"; throw std::out_of_range("playlist"); }
        return tracks[i];
    }
    const std::string& operator[](int i) const {
        if (i < 0 || i >= size()) { std::cerr << "Index " << i << " out of
range\n"; throw std::out_of_range("playlist"); }
        return tracks[i];
    }
};
int main() {
```

```
    Time t(59 * 60 + 58);
    Time a = t++, b = ++t;
    std::cout << a << ' ' << b << ' ' << t << '\n';
    ++t;
    std::cout << t << " " << std::boolalpha << (a < t) << '\n';
    Playlist p; p.add("Track A"); p.add("Track B");
    std::cout << p[0] << '\n';
    try { p[-1]; } catch (const std::out_of_range&) {}
    try { p[p.size()]; } catch (const std::out_of_range&) {}
}
```

Expected output:

```
0:59:58 1:00:00 1:00:00
1:00:01 true
Track A
```

Work out and verify: start 0:59:58; `a = t++` → a = 0:59:58, t = 0:59:59; `b = ++t` → t = b = 1:00:00. So the first line is `0:59:58 1:00:00 1:00:00`, then `1:00:01 true`, then `Track A`, then two lines `Index -1 out of range` and `Index 2 out of range` on stderr (59:59 → 1:00:00 roll-over confirmed).

Design & Analysis

- **Prefix vs postfix:** the compiler calls `operator++()` for `++t` and `operator++(int)` for `t++`; the unused `int` is only a dummy that distinguishes the two signatures.
- **Postfix returns a copy and is slower:** it must save the old value, increment, then return the saved copy; prefix increments and returns a reference, no copy.
- **const and non-const `operator[]`:** the non-const version returns a modifiable reference (`p[0] = "New"`); the const version lets a `const Playlist&` be indexed read-only. Without it, const playlists cannot be read.
- **Out of range:** report the error and refuse (throw an exception); never read outside the array.

Experimental Investigation

Test	Result
After `a = t++`, `b = ++t`	a = old value, b = new value (see output)
10,000,000 prefix increments	-O0: _____ ms, -O2: _____ ms
10,000,000 postfix increments	-O0: _____ ms, -O2: _____ ms
`playlist[-1]`, `playlist[size]`	error printed, exception thrown
Roll-over 0:59:59 → 1:00:00	correct

Conclusion

For class types `++i` avoids creating and copying a temporary; `i++` costs a copy that the optimizer may not remove when the copy constructor is non-trivial.

Predict the Output

My predicted output (before running): `5 7 7`

Explanation: `c++` returns the old copy (`a.v = 5`) and then `c.v` becomes 6; `++c` increments to 7 and returns a reference to `c`, so `b.v = 7`; `c.v = 7`. `a` and `b` differ because postfix returns the value *before* the increment and prefix the value *after*. The **postfix** version makes a copy (`Counter old = *this;`).

Alternative Concept – `at(i)` or unchecked `[]`

```
std::string& at(int i) { return tracks.at(i); }   // throws std::out_of_range automatically
std::string& operator[](int i) { return tracks[i]; } // unchecked, like a built-in array
```

Time per access O(1) in all versions; checked access adds one extra comparison (two with the `< 0` test). Space O(1). Measure 10,000,000 accesses of each: checked _____ ms, unchecked _____ ms (usually a few percent difference).

Q6 – The Rules of Operator Overloading (7 Marks)

Implementation (`Task06/text.cpp`)

```
#include <cstring>
#include <iostream>
class Text {
    char* data;
    std::size_t len;
public:
    static int copies;
    Text(const char* s = "") : len(std::strlen(s)), data(new
char[std::strlen(s) + 1]) { std::strcpy(data, s); }
    Text(const Text& o) : data(new char[o.len + 1]), len(o.len)
{ std::strcpy(data, o.data); ++copies; }
    Text& operator=(const Text& o) {
        if (this != &o) {                       // self-assignment guard
            char* t = new char[o.len + 1];
            std::strcpy(t, o.data);
            delete[] data; data = t; len = o.len; ++copies;
        }
        return *this;
    }
    ~Text() { delete[] data; }
    friend Text operator+(const Text& a, const Text& b) {
        char* buf = new char[a.len + b.len + 1];
        std::strcpy(buf, a.data); std::strcat(buf, b.data);
        Text r(buf); delete[] buf; return r;
    }
    friend bool operator==(const Text& a, const Text& b) { return
std::strcmp(a.data, b.data) == 0; }
    friend bool operator!=(const Text& a, const Text& b) { return !(a ==
b); }
    friend std::ostream& operator<<(std::ostream& os, const Text& t) { return
```

```
os << t.data; }
};
int Text::copies = 0;
int main() {
    Text a("Hello "), b("SMS "), c("Gateway");
    Text d = a + b + c;
    std::cout << d << '\n' << std::boolalpha << (d == Text("Hello SMS
Gateway")) << '\n';
    d = d;                                          // self-assignment
    Text e;                                         // empty message
    std::cout << '[' << e << "] " << (d != e) << " copies=" << Text::copies
<< '\n';
}
```

Expected output (first two lines):

```
Hello SMS Gateway
true
```

The third line shows `[] true copies=N`; `N` depends on whether your compiler elides the returned temporaries (try `-fno-elide-constructors` to see the maximum). Record N for `a + b + c`: ____

Design & Analysis

- **Operators that cannot be overloaded:** `::` (scope: left side is a name, not an object), `.` and `.*` (member access must keep fixed meaning, otherwise class internals become unpredictable), `?:` (cannot control short-circuit evaluation of three operands), `sizeof` (evaluated at compile time).
- **Rules:** precedence and associativity cannot change; arity cannot change (a binary operator stays binary); new operator symbols cannot be invented; at least one operand must be a user-defined type (you cannot redefine `int + int`).
- **`operator=` must be a member:** the language requires `=`, `[]`, `()`, `->` to be members, so that the left operand is always an lvalue object of the class.
- **When a friend:** when a non-member operator needs private data (`<<`, `==` here) or the left operand is not our class (`std::ostream`, or `100 + m`).

Experimental Investigation

Test	Result
Copies for `a + b + c`	____ (static counter)
`-fsanitize=address`	expect clean; typical bugs found here: missing `+1` for the null terminator, missing `delete[]`
100,000 concatenations: `Text` vs `std::string`	Text: ____ ms, std::string: ____ ms
Self-assignment `d = d`	unchanged because of the guard
Empty message and long message (e.g. 10,000 chars)	works, length 0 handled by the `+1` for `'\0'`

Conclusion

Writing `Text` showed how the Rule of Three, deep copying and self-assignment safety actually work. I would still write such a class when I need special memory control (custom allocator, fixed buffer, embedded system without the standard library) or when teaching.

Alternative Concept – `std::string` or `+` via `+=`

```
Text operator+(Text a, const Text& b) { a += b; return a; }   // a is taken by value (one copy, then appended)
```

For `a + b + c` with lengths n1, n2, n3: the buffer version above copies (n1+n2) characters for the first `+` and then (n1+n2+n3) for the second, so total characters copied about 2n1 + 2n2 + n3; temporary space about (n1+n2) + (n1+n2+n3). The by-value + `+=` version copies n1 once, appends n2, then appends n3 onto the temporary, so about n1 + n2 + n3 characters copied if `+=` grows the buffer, and fewer temporaries. `std::string` behaves like the second version (and has small-string optimization). Time is O(n1+n2+n3) in all cases.

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the Problem (2)

Data: real part `re` and imaginary part `im`, both `double`. **Operations:** add (series), multiply and divide (parallel), subtract, compare, print as `a + bj`, magnitude √(re²+im²). **Constructors:** default `Complex()` = 0+0j; from real part `Complex(R)` = R+0j (resistor); from both `Complex(re, im)` (inductor/capacitor). One constructor with default arguments covers all three. **Measurable requirements:**

1. Z(R=10 Ω) prints `10.00 + 0.00j`.
2. Series R=10, L=20 mH, C=100 µF at ω=1000 rad/s gives `10.00 + 10.00j`, |Z| = 14.14 Ω.
3. Parallel R and L gives `8.00 + 4.00j`.
4. Division by 0+0j must be reported as an error (exception), never produce `inf/nan`.

Stage 2 – Design (3)

Operator	Form	Formula
`+`	member, const	(a+bj)+(c+dj) = (a+c) + (b+d)j
`-`	member, const	(a−c) + (b−d)j
`*`	member, const	(ac−bd) + (ad+bc)j
`/`	member, const	((ac+bd) + (bc−ad)j) / (c²+d²); throws if c²+d² = 0
`==`	member, const	re equal and im equal (compare with a small tolerance in practice)
`<<`	friend (non-member)	prints `re + imj` or `re - imj`

`+ - * / ==` are members; `<<` is a friend because its left operand is `std::ostream`. Making `+ - * /` friends instead would also allow `5 + z` (see Alternative).

Stage 3 – Implement and Verify (3) (`Task07/complex.cpp`)

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>
class Complex {
    double re, im;
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}
    Complex operator+(const Complex& o) const { return Complex(re + o.re, im
+ o.im); }
    Complex operator-(const Complex& o) const { return Complex(re - o.re, im
- o.im); }
    Complex operator*(const Complex& o) const { return Complex(re * o.re - im
* o.im, re * o.im + im * o.re); }
    Complex operator/(const Complex& o) const {
        double d = o.re * o.re + o.im * o.im;
        if (d == 0) throw std::domain_error("division by zero impedance");
        return Complex((re * o.re + im * o.im) / d, (im * o.re - re * o.im) /
d);
    }
    bool operator==(const Complex& o) const { return re == o.re && im ==
o.im; }
    double magnitude() const { return std::sqrt(re * re + im * im); }
    friend std::ostream& operator<<(std::ostream& os, const Complex& z) {
        os << std::fixed << std::setprecision(2) << z.re << (z.im < 0 ? " - "
: " + ") << std::fabs(z.im) << 'j';
        return os;
    }
};
int main() {
    const double w = 1000, L = 0.02, C = 100e-6, R = 10;
    Complex ZR(R), ZL(0, w * L), ZC(0, -1 / (w * C));
    Complex series = ZR + ZL + ZC;
    std::cout << "Series   : " << series << "  |Z| = " << series.magnitude()
<< '\n';
    Complex rl = (ZR * ZL) / (ZR + ZL);
    std::cout << "Parallel RL : " << rl << "  |Z| = " << rl.magnitude() << '\
n';
    Complex rlc = (rl * ZC) / (rl + ZC);
    std::cout << "Parallel RLC: " << rlc << "  |Z| = " << rlc.magnitude() <<
'\n';
    try { Complex bad = ZR / Complex(0, 0); std::cout << bad; }
    catch (const std::domain_error& e) { std::cout << "Error: " << e.what()
<< '\n'; }
}
```

Expected output:

```
Series   : 10.00 + 10.00j  |Z| = 14.14
Parallel RL : 8.00 + 4.00j  |Z| = 8.94
```

```
Parallel RLC: 8.00 - 4.00j  |Z| = 8.94
Error: division by zero impedance
```
Hand verification (ω = 1000, X_L = ωL = 20 Ω, X_C = 1/(ωC) = 10 Ω):

- Series: 10 + j20 − j10 = **10 + j10**; |Z| = √200 = 14.14.
- Parallel R‖L: (10·j20)/(10+j20) = j200(10−j20)/500 = (4000 + j2000)/500 = **8 + j4**; |Z| = √80 = 8.94.
- Add C: (8+j4)(−j10)/(8+j4−j10) = (40 − j80)/(8 − j6) = (40−j80)(8+j6)/100 = (800 − j400)/100 = **8 − j4**; |Z| = 8.94.

Stage 4 – Defend the Design (2)

- **vs `std::complex<double>`:** the standard class already has all operators, `abs()`, `polar()`, mixed real/complex arithmetic and well-tested numerics (e.g. robust division). Ours is simpler, teaches the formulas and gives a clear error on division by zero.
- **Why each piece:** one default-argument constructor builds 0, R and R+jX; `+` for series; `*` and `/` for parallel; `-` for differences; `==` for tests; `<<` for printing; `magnitude()` for |Z|.
- **Remaining limitation:** `==` on doubles is exact (needs a tolerance); no `operator+=` or mixed `double + Complex` support; division uses the plain formula, which can overflow/lose accuracy for very large or tiny values.

Alternative Concept – `std::complex<double>`

```
#include <complex>
std::complex<double> ZR(10, 0), ZL(0, 20), ZC(0, -10);
auto series = ZR + ZL + ZC; auto par = (ZR * ZL) / (ZR + ZL);
```
Time per operation O(1) in both; space is two doubles in both (16 bytes). `std::complex` uses a scaled division algorithm (better accuracy for extreme values). Dividing by zero does *not* throw: it returns `inf/nan`, so the program must check the divisor itself (`if (z == std::complex<double>(0))`), whereas our class handles it inside `operator/`.

Submission Checklist

- GitHub repository: https://github.com/Sangeetthangaraj/DSA0104---object-oriented-programming-with-c-
- One folder per task (Task01 … Task07) with source files and output screenshots; `README.md` with name, register number, compiler version, exact g++ command and a one-line result per task.
- At least one commit per task (progressive commits).
- Fill in: compiler version, all _____ timings, sanitizer outputs, and screenshots before submitting.